

AegisSync — Secure SaaS Integration Architecture

GitHub + Notion + Payment Gateway on AWS

Disclaimer

AegisSync is a **demo architecture** designed to demonstrate secure, scalable, and production-grade SaaS patterns using AWS and third-party integrations.

1. What is this Project?

AegisSync is a **secure hybrid SaaS integration platform** that connects:

- GitHub (developer workflows)
- Notion (workspace/data sync)
- Payment Gateway (transactions & subscriptions)

It is designed as a **multi-tenant, security-first backend system** with:

- Hybrid communication (synchronous + asynchronous)
 - Strong IAM isolation
 - Secure webhook handling
 - Event-driven scalability
-

2. Why this Project?

Modern SaaS systems:

- Depend heavily on third-party APIs
- Handle sensitive user and payment data
- Require both real-time and background processing

Most systems fail due to:

- Poor IAM practices

- Weak webhook validation
- Incorrect sync vs async usage
- Overexposed infrastructure

👉 AegisSync demonstrates how to design it correctly.



3. What Problem Does It Solve?

Problem 1: Insecure Integrations

- Unverified API calls
- Token exposure

Problem 2: Incorrect Communication Model

- Everything synchronous → slow
- Everything asynchronous → inconsistent UX

Problem 3: Webhook Exploits

- Fake callbacks
- Replay attacks

Problem 4: Weak Network Isolation

- Publicly exposed services
- Overuse of NAT

Problem 5: Payment Fraud

- Client-side validation
 - Missing verification
-



4. Benefits

- 🛡️ Security-first architecture
- ⚡ Hybrid performance model
- 🧩 Modular and extensible
- 🛡️ Strong isolation boundaries

-  Secure payment workflows
 -  Internet-less internal communication
-

5. Use Cases

- SaaS productivity platforms
 - Developer tooling integrations
 - Subscription-based systems
 - Workflow automation tools
 - API-first startups
-

6. Components Used

Edge Layer

- API Gateway
- AWS WAF
- AWS Shield

Identity & Authentication

- IAM Roles
- AWS STS
- Amazon Cognito

Compute

- AWS Lambda

Storage

- Amazon S3
- DynamoDB / RDS

Secrets & Encryption

- AWS Secrets Manager

- AWS KMS

Networking

- VPC
- Public & Private Subnets
- NAT Gateway
- VPC Interface Endpoints (PrivateLink)
- Gateway Endpoints (S3, DynamoDB)

Integrations

- GitHub API + Webhooks
- Notion API
- Payment Gateway

Monitoring

- CloudWatch Logs
- Metrics & Alerts

7. User Flow (Mapped to Diagram Steps 1–7)

◆ Step 1 — User Entry

User initiates request from frontend.

◆ Step 2 — WAF Filtering

Request passes through WAF
→ Blocks SQL injection, XSS, bot traffic

◆ Step 3 — Authentication (Cognito)

User identity verified via JWT/OAuth

👉 Synchronous (critical path)

◆ Step 4 — API Gateway Routing

Routes validated requests to backend services

◆ Step 5 — Load Distribution

Traffic routed to appropriate services (GitHub / Notion paths)

◆ Step 6 — Compute Execution

Lambda processes:

- Business logic
- Integration triggers

◆ Step 7 — Data Processing Layer

◆ Sync Path (Critical)

- Authentication
- Payment processing

👉 Immediate response required

◆ Async Path (Background)

- SQS → Worker Lambda → External APIs
- GitHub indexing
- Notion synchronization

👉 Decoupled, scalable, failure-tolerant

Payment Flow (Synchronous)

1. User initiates payment
2. Request sent to gateway
3. Gateway processes
4. Webhook callback received
5. Signature verified

6. Database updated

👉 Strictly synchronous validation for integrity

8. Deployment Architecture (Hybrid Model)

◆ Primary Path (Synchronous)

• Cognito → API Gateway → ALB → Lambda

Used for:

- Authentication
- Payments
- Critical APIs

👉 Ensures real-time response

◆ Worker Path (Asynchronous)

• SQS → Background Lambda → GitHub / Notion APIs

Used for:

- Data sync
- Event processing
- High-volume workloads

👉 Improves scalability and resilience

9. Security Considerations

9.1 IAM (Least Privilege)

- Separate IAM roles per service
 - No wildcard permissions
 - Strict resource-level access
-

9.2 STS (Temporary Credentials)

- Short-lived credentials
- Reduces attack window
- Used for role assumption

9.3 KMS (Encryption)

Encrypts:

- S3
- Database
- Secrets

 Protects against data exposure

9.4 Secrets Manager

Stores:

- API tokens
- Payment secrets
- Enables automatic rotation

9.5 WAF Protection

Blocks:

- SQL Injection
- XSS
- Bots
- Rate abuse

9.6 Network Security (VPC Deep Dive)

Private Subnet Isolation

Private Subnet Isolation

- Lambda and DB are isolated
- No direct internet exposure

Interface Endpoints (PrivateLink)

Used for:

- KMS
- Secrets Manager

👉 Traffic remains within AWS network

👉 No NAT → **internet-less communication**

Gateway Endpoints

- S3
- DynamoDB

👉 Secure internal access

NAT Gateway

- Used only for external APIs (GitHub / Notion)



9.7 Webhook Security

Threats:

- Fake webhook
- Replay attack

Mitigation:

- HMAC signature validation
- Timestamp validation
- Idempotency keys



9.8 Payment Security

- No card data stored
- Gateway handles PCI compliance

- Server-side validation only

9.9 Logging & Monitoring

Logs:

- API logs
- Auth logs
- Payment logs
- Integration logs

Alerts:

- Suspicious activity
- Failed validations

9.10 Attack Prevention Summary

Attack	Prevention
SQL Injection	WAF
XSS	Encoding + WAF
Replay Attack	Timestamp + nonce
Token Leak	STS + Secrets Manager
Fake Webhook	Signature validation
DDoS	AWS Shield

10. Additional Enhancements

- Multi-account tenant isolation
 - RBAC system
 - CI/CD with security scanning
 - SIEM integration
 - Per-tenant rate limiting
-



11. Final Thoughts

AegisSync demonstrates:

- Hybrid SaaS architecture
- Real-world integration design
- Strong security posture
- Cloud-native scalability